

React Conditional Rendering Hands-On Lab — Solution

Building the Flight Ticket Booking App ("ticketbookingapp") with Guest and User Views

Objectives

- Explain conditional rendering in React
- Define element variables
- Explain how to prevent components from rendering

In this hands-on lab, you will learn how to:

- Implement conditional rendering in React applications

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

Create a React application named "ticketbookingapp" where a guest user can browse a page showing flight details, but only a logged-in user can book tickets. The Login and Logout buttons must accordingly display different pages: once the user logs in, the User page (with the booking option) should be displayed; when the user clicks Logout, the Guest page should be displayed again.

Solution — Step by Step

Step 1: Create the React project "ticketbookingapp"

Open a terminal and run the following command to scaffold the project:

```
npx create-react-app ticketbookingapp
```

Once the setup finishes, navigate into the project folder:

```
cd ticketbookingapp
```

Step 2: Open the application in VS Code

```
code .
```

Step 3: Create the flight data

Add a new file named FlightsMock.js in the src folder with some sample flight details:

```
const FlightsMock = [  
  { id: 1, airline: 'Air Horizon', from: 'Delhi', to: 'Mumbai', price: 4500 },  
  { id: 2, airline: 'SkyBlue Airways', from: 'Bengaluru', to: 'Chennai', price:  
    2800 },  
]
```

```
{ id: 3, airline: 'CloudJet', from: 'Kolkata', to: 'Hyderabad', price: 5200 },
];

export default FlightsMock;
```

Step 4: Create the GuestPage component

Add a new file named GuestPage.js. It displays the flight details but does not allow booking:

```
import React from 'react';
import FlightsMock from './FlightsMock';

function GuestPage() {
  return (
    <div className="guest-page">
      <h2>Available Flights</h2>
      <p>Please log in to book a ticket.</p>
      <ul>
        {FlightsMock.map(flight => (
          <li key={flight.id}>
            {flight.airline}: {flight.from} &#594; {flight.to}
            (&#8377;{flight.price})
          </li>
        ))}
      </ul>
    </div>
  );
}

export default GuestPage;
```

Step 5: Create the UserPage component

Add a new file named UserPage.js. It displays the same flight details, but with a Book Now button available for the logged-in user:

```
import React from 'react';
import FlightsMock from './FlightsMock';

function UserPage() {
  return (
    <div className="user-page">
      <h2>Available Flights</h2>
      <ul>
        {FlightsMock.map(flight => (
          <li key={flight.id}>
            {flight.airline}: {flight.from} &#594; {flight.to}
            (&#8377;{flight.price})
            <button onClick={() => alert('Ticket booked on ' + flight.airline)}>
              Book Now
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default UserPage;
```

```

        </button>
      </li>
    )}
  </ul>
</div>
);
}

export default UserPage;

```

Step 6: Create the LoginButton and LogoutButton components

Add a file named LoginButton.js:

```

import React from 'react';

function LoginButton({ onClick }) {
  return <button onClick={onClick}>Login</button>;
}

export default LoginButton;

```

Add a file named LogoutButton.js:

```

import React from 'react';

function LogoutButton({ onClick }) {
  return <button onClick={onClick}>Logout</button>;
}

export default LogoutButton;

```

Step 7: Track login state and conditionally render in App.js

In App.js, keep an isLoggedIn flag in state. Use element variables to decide which page and which button to render, rather than duplicating markup with nested ternaries throughout the JSX:

```

import React, { useState } from 'react';
import './App.css';
import GuestPage from './GuestPage';
import UserPage from './UserPage';
import LoginButton from './LoginButton';
import LogoutButton from './LogoutButton';

function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  const handleLogin = () => setIsLoggedIn(true);
  const handleLogout = () => setIsLoggedIn(false);

  // Element variables: decide once which button and which

```

```

// page to render, then reuse the variables in the JSX below
let authButton;
if (isLoggedIn) {
  authButton = <LogoutButton onClick={handleLogout} />;
} else {
  authButton = <LoginButton onClick={handleLogin} />;
}

let page;
if (isLoggedIn) {
  page = <UserPage />;
} else {
  page = <GuestPage />;
}

return (
  <div className="App">
    <header>
      <h1>Flight Ticket Booking</h1>
      {authButton}
    </header>
    {page}
  </div>
);
}

export default App;

```

Step 8: Build and run the application

```
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

On load, `isLoggedIn` is false, so the Login button and the Guest page (flight details only, no booking option) are rendered. Clicking Login sets `isLoggedIn` to true, which causes the element variables to switch: the Logout button and the User page (with the Book Now option) are rendered instead. Clicking Logout sets `isLoggedIn` back to false, and the Guest page is displayed again — demonstrating how conditional rendering can show or completely prevent a component from rendering based on the application's state.

Result

A React application named "ticketbookingapp" was successfully created with a `GuestPage` component (flight details only) and a `UserPage` component (flight details plus booking). The `App` component tracks an `isLoggedIn` state value and uses element variables to conditionally render either the Login or Logout button and either the Guest or User page. The application was verified by running it locally and confirming that logging in and out correctly switches between the two views at `http://localhost:3000`.